1. A arquitetura de von Neumann é caracterizada por CPU (composta por unidade lógica e aritmética, unidade de controle, registradores e contador de programas – PC), memória (RAM onde são armazenados dados e programas), e periféricos de I/O.	F
2. Na arquitetura de von Neumann o processamento é sequencial e segue o ciclo: busca – decodificação – busca de operandos – execução. O carregador ao carregar o programa na memória atribui a PC o valor do **entry point**, passando PC a apontar para uma instrução e esse ciclo é iniciado.	V
3. A execução é rápida na arquitetura de von Neumann porque dados e instruções são armazenados em blocos de memória adjacentes devido ao fenômeno de localidade de referência	V
4. A redução do gargalo de von Neumann entre CPU e periféricos de I/O pode ser obtida com o uso de memória cache e processadores **pipelining** com múltiplos estágios.	F
5. Código absoluto foi utilizado no início da computação com sistemas monousuário e não é mais utilizado devido a que os sistemas atuais são multiprogramados ou multiprocessados.	F
6. O advento do código relocável permitiu o uso de bibliotecas de código objeto como é o caso do **bytecode** Java que roda em qualquer computador com uma JVM Java instalada.	V
7. Ao se ligar um computador, pressionando o botão de **power on**, um carregador absoluto armazenado em memória volátil é ativado e inicia todo o processo de carga do sistema operacional.	F
8. Um montador é um programa que converte um programa em código Assembly em um código objeto relocável (código de máquina), se o programa Assembly estiver correto, e cria um arquivo binário com código de máquina relocável, contendo uma tabela (ou lista) de endereços relocáveis e uma lista de símbolos não resolvidos pelo montador.	V
9. Do ponto de vista lógico, em geral um montador é estruturado em um pré-processamento (retira comentários, expande macro e faz a montagem condicional), passo 1 (análises léxica, sintática e semântica) e síntese de código (otimização e geração de código absoluto).	NS
10. As otimizações algorítmicas de código que podem ser realizadas na síntese do código são de tempo de execução ou da alocação de memória	V
11. Um ligador é um programa que, a partir do conjunto de arquivos contendo códigos objetos relocáveis, resolve as referências externas e gera um arquivo contendo os códigos objetos ligados e com indicação dos endereços relocáveis. Esse arquivo, união dos códigos objetos, é executável.	V
12. A função básica do ligador é ligar os módulos e resolver as referências externas, seguindo as seguintes etapas: ler e relocar de forma estática o módulo objeto P; atualizar a tabela global de símbolos com os símbolos internos de P que podem ser referenciados em outros módulos; concatenar P aos módulos anteriores; e resolver as referências externas em P.	V

13. Os ligadores podem ser apenas dos tipos **Linkage Editor** (gera um arquivo binário executável que pode ser utilizado novamente), **Linking Loader** ou **Binder** (carrega diretamente o código gerado na memória sem gerar um arquivo binário que possa ser reutilizado), ou carregador de função de biblioteca de ligação dinâmica (tipo DLL).	V
14. O carregador de relocação estática lê o tamanho do programa a carregar; solicita ao sistema operacional (S.O.) a alocação de um bloco contíguo de memória com tamanho suficiente para conter o programa; carrega o programa a partir do endereço dado pelo S.O. relocando os endereços relocáveis; e inicializa o PC com o **entrypoint** do programa.	V
15. Um escalonamento de processos adequado para um S.O. multiprocessado é o **round-robin** pois permite evitar **starvation**	V
16. Sistemas de escalonamento procuram maximizar o **throughput** e minimizar o **turnaround time** e o **response time.** Por isso um bom escalonador deve ser não preemptivo.	F
17. Para maximizar o **throughput** e evitar **deadlock** um processo só pode entrar na fila de prontos se já estiver com todos os recursos que necessitar.	NS
18. Região crítica é o trecho do código que objetiva garantir a exclusão mútua de processos.	V
19. Em um S.O. do tipo **batch** a melhor gerência de memória real é paginação-segmentação.	NS
20. Em um S.O. do tipo time **sharing** como as transações são CPU intensiva, a melhor gerência de memória real é de partições.	NS
21. Gerência de memória baseada em segmentos minimiza a fragmentação interna mas favorece a proteção de código e a fragmentação externa.	F

21. Gerência de memória baseada em segmentos minimiza a fragmentação interna mas favorece a proteção de código e a fragmentação externa.	F

22. Se um processo que estiver de posse da CPU solicitar um acesso a uma página não disponível na memória será criado um programa de canal para carregar a página em uma **page frame** e o processo ficará bloqueada.	V

Qual o mecanismo que será utilizado para retirar o processo da fila de bloqueado e o introduzir na fila de prontos quando a requisição de acesso a disco tiver sido atendida?
Escalonador.

Responda com apenas uma palavra.

Justificativas para as questões respondidas como falso
Questão 01 – A memória RAM não necessariamente será a memória utilizada.
Questão 04 – Além dos citados, necessita dos métodos DMA, bufferização e blocagem.
Questão 05 – É utilizado no boot
Questão 07 – Um carregador absoluto armazenado em memória **Não** volátil.
Questão 16 - Um bom escalonador deve ser preemptivo.
Questão 21 – Aumenta a fragmentação interna.

Parte 2 - questões discursivas

1. Caracterize o que é (0,5) e o que causa localidade de referência (0,5). (1,0)
   Resposta: A localidade de referência é a organização da memória de forma continua, onde o programa é colocado em blocos sequenciais de memória, sendo assim as instruções estarão lado a lado.

2. Quais são os tipos de carregadores existentes? (0,5) Explique em que instante se dá a relocação dos endereços (0,25) e o que é carregado em cada um dos tipos de carregador que citar (0,25).(1,0)
   Resposta: Temos o carregador Absoluto e o carregador de relocação.

3. Apresente um exemplo e explique como é realizado o mapeamento de um endereço lógico em um endereço real em um ambiente paginado. (1,0)
   Resposta: Não sei
4. Coloque-se na posição de um projetista de sistema operacional. Informe a aplicação característica (**batch**, tempo real, **time sharing**, rede ou distribuído), forma de carregamento de programas na memória (contíguos ou não), tipo (monoprogramado, multiprogramado ou multiprocessado), e estrutura (hierárquica, máquinas virtuais, multiprocessados, distribuídos ou orientados a objetos). Apresente e justifique a política de processos adequada (0,5) indicando como ela evita **deadlock** (0,5) e **starvation** (0,5), e a política de gerência de memória real. (0,5) (2,0)
   Resposta: Não Sei